

Theoretical Computer Science

Cyclic Proofs and Size-Change Termination

--Manuscript Draft--

Manuscript Number:	TCS-D-24-00949
Article Type:	Regular Paper (10 - 40 pages)
Section/Category:	Logic, semantics and theory of programming
Keywords:	Cyclic proofs; Size-change termination; Intuitionistic logic
Abstract:	A cyclic proof is a regular infinite proof tree with a global condition, and it is known that, under arithmetic, cyclic proofs with inductive definitions are equivalent to proofs with induction. A way to show this equivalence is by constructing a well-founded relation on tuples of natural numbers and then transforming a cyclic proof into a proof by well-founded induction with the relation. In this paper, we see the connection between cyclic proofs and size-change termination (SCT) of programs, and it provides a new well-founded relation for proof transformation.

Cyclic Proofs and Size-Change Termination

Mirai Ikebuchi

^a*Graduate School of Informatics, Kyoto University, Yoshida Honmachi,
Sakyo-ku, Kyoto, 606-8501, Kyoto, Japan*

Abstract

A cyclic proof is a regular infinite proof tree with a global condition, and it is known that, under arithmetic, cyclic proofs with inductive definitions are equivalent to proofs with induction. A way to show this equivalence is by constructing a well-founded relation on tuples of natural numbers and then transforming a cyclic proof into a proof by well-founded induction with the relation. In this paper, we see the connection between cyclic proofs and size-change termination (SCT) of programs, and it provides a new well-founded relation for proof transformation.

Keywords: Cyclic proof, Size-change termination, Intuitionistic logic

1. Introduction

Cyclic proof systems are proof systems in which proofs are regular infinite trees with the *global trace condition* which ensures that any infinite path in the tree has an “infinite descent”. In [1], [2], the cyclic proof system for first-order logic with Martin-Löf’s inductive definition [3] is investigated. Although cyclic proofs can simulate proofs by induction, Berardi and Tatsuta proved that there exists a proposition that is provable cyclically but not by induction ([4]). On the other hand, if the system includes arithmetic, cyclic proof and proof by induction are equivalent. The equivalence is first proved in [5] for his setting of the cyclic proof system, *Cyclic Arithmetic*, in classical first-order logic, but his Cyclic Arithmetic does not cover general Martin-Löf’s inductive definitions. For Brotherston’s original setting of the cyclic proof system, the equivalence is proved by Berardi and Tatsuta for both classical [6] and intuitionistic [7] logic.

Cyclic proofs are applied to the field of program verification like termination checking of imperative pointer programs [8], program synthesis [9], and model checking [10]. Despite its practical importance, the existing proofs of equivalence can be hard to understand for non-experts. The proof in [5] heavily relies on reverse mathematics. The proof in [6] requires less technical knowledge; they construct a well-founded relation¹ and transform a cyclic proof

¹Berardi and Tatsuta proved that their relation has the principle of well-founded induction, not well-foundedness, but here we are not distinguishing them for simplicity.

tree into a (finite) proof tree with well-founded induction. The intuitionistic logic version of equivalence [7] is also shown by proof transformation. However, the proof of well-foundedness of the constructed relation is more complicated and takes 7 pages.

In this paper, we will see that a well-founded relation for proof transformation can be constructed from *size-change graphs* associated with the given cyclic proof. Size-change graphs are used to prove the termination of programs, introduced in [11]. A program is abstracted into a set of *size-change graphs*, each of which is a directed graph that describes how the values of parameters change in a function call. Then, the size-change termination (SCT) condition is that every infinite sequence of function calls has infinite descent, that is, there is a parameter that strictly decreases infinitely many times. The connection between cyclic proofs and SCT has been informally understood by experts of cyclic proof (e.g., [8]) but has not been described formally.

Not only is there a connection, but we will see that a proof transformation can be done with the lexicographic order on tuples of natural numbers as a well-founded relation and a *ranking function*. For a recursive program, a ranking function is a function ρ taking the program's arguments and returning a tuple of natural numbers such that the value of ρ strictly decreases in each recursive call. It is proved that any program with a set of size-change graphs with SCT condition has a ranking function in [12]. We use this fact to transform cyclic proofs into proofs with induction. Since we use the lexicographic order on tuples of natural numbers, there is no need for a long proof of well-foundedness.

The next section explains cyclic proofs informally, not as formal proofs. We will see how a cyclic proof can be transformed into a proof by induction and how it corresponds to SCT. We give an introduction to size-change termination in Section 3 and an introduction to cyclic proof in Section 4. In Section 5, we define a set of size-change graphs with SCT condition from a $\text{CLJID}^\omega + \text{HA}$ proof and then prove that we can replace Berardi-Tatsuta's well-founded relation with our new relation $\rho(-) > \rho(-)$. In fact, the proof is mostly obvious from the results of [12] and [7]. This means that the equivalence proof in [7] is simplified, at least for those already familiar with size-change termination. Even for those who are not familiar with size-change termination, the author believes that showing the connection between cyclic proofs and SCT helps to understand the equivalence between cyclic proofs and proofs by induction.

2. Case study

Before getting into the formal definition of cyclic proof, we see simple examples of cyclic proofs in English, not as formal proofs, and how we can transform it into a proof by induction.

2.1. Ranking function

Suppose that we have a constant symbol 0 and a unary function symbol s . We consider *inductive predicates*. For example, the inductive predicate Nx is defined by rules

$$\frac{}{N0} \quad \frac{Nx}{N(sx)}$$

These rules mean that Nx holds iff x is a natural number built from s and 0. More precisely, the above rules means that we assume the following:

- $N0$ holds.
- For any x , if Nx holds, then $N(sx)$ holds.
- For any x , if Nx holds, either (i) $x = 0$ or (ii) there exists x' such that $x = sx'$ and Nx' holds.

We assume that we have the binary function symbols $+$, $\times$, and assume Peano axioms like $Nx \rightarrow sx \neq 0$. Also, we define $x < y$ by $\exists z. x + sz = y$.

Suppose that we have a (non-inductive) binary predicate symbol Q and three axioms

$$\begin{aligned} h_0 : Q(0, 0), \quad h_1 : \forall x y. Nx \wedge Ny \wedge Q(sy, y) \rightarrow Q(x, sy), \\ h_2 : \forall x y. Nx \wedge Ny \wedge Q(y, x) \rightarrow Q(sx, y). \end{aligned}$$

We cyclically prove the following proposition.

Proposition. $\forall x y. Nx \wedge Ny \rightarrow Q(x, y)$.

Proof. Suppose Nx and Ny and show $Q(x, y)$. (*) Since Nx holds, we have two cases:

1. Case $x = 0$: Since Ny holds, we have two cases:
 - (a) Case $y = 0$: By h_0 , we have $Q(0, 0)$.
 - (b) Case $y = sy'$: by h_1 , it is enough to show $Q(sy', y')$. Go to (*), substituting sy' to z and y' to y .
2. Case $x = sx'$: by h_2 , it is enough to show $Q(y, x')$. Go to (*), substituting y to x , x' to y . □

Here, we have a recursion, so our “proof” forms an infinite tree. However, this proof has an “infinite descent” in the following sense. Each infinite path of the tree has an infinite number of case analyses on Nx or Ny . This condition — every infinite path has infinite number of case analyses on inductive predicates — is called the *global trace condition*. In Brotherston’s cyclic proof system CLKID^ω , a proof is defined as a regular infinite pre-proof tree with the global trace condition. (We give a formal definition in Section 4.) It may be helpful to see that we can encode our proof as the following dependently-typed program in

```

1
2
3
4   Q : Nat -> Nat -> Prop
5   h0 : Q 0 0
6   h1 : forall (x y : Nat) -> Q (s y) y -> Q x (s y)
7   h2 : forall (x y : Nat) -> Q y x -> Q (s x) y
8
9
10  f : forall (x y : Nat) -> Q x y
11  f 0 0 = h0
12  f 0 (s y') = h1 0 y' (f (s y') y')
13  f (s x') y = h2 x' y (f y x')
14
15
16

```

Figure 1: A dependently-typed program corresponding to the cyclic proof

Agda-like syntax (Figure 1). where we use the type **Nat** of natural numbers built from 0 and **s** instead of directly encoding the predicate N . Intuitively, the fact that our infinite proof has an infinite descent corresponds to the fact that the function **f** terminates.

We assume the induction principle for natural numbers: For a predicate Fx , to prove that Nx implies Fx for any x , it suffices to prove (i) $F0$ and (ii) for any x' with Nx' , if Fx' holds, then $F(sx')$ holds. This principle implies well-founded induction for $<$.

Then, let us transform our cyclic proof of $\forall x y. Nx \wedge Ny \rightarrow Q(x, y)$ into a proof by induction.

Proof. Let $\rho(x, y) = \max(2x - 1, 2y)$. We prove $\forall x y. Nx \wedge Ny \rightarrow Q(x, y)$ by well-founded induction on $\rho(x, y)$. Suppose the statement holds for any x', y' with $\rho(x', y') < \rho(x, y)$. Since Nx holds, we have two cases:

1. Case $x = 0$: Since Ny holds, we have two cases:
 - (a) Case $y = 0$: By h_0 , we have $Q(0, 0)$.
 - (b) Case $y = sy'$: Since $\rho(sy', y') = \max(2y' + 1, 2y') < \max(2x - 1, 2y' + 2) = \rho(x, sy')$, by the induction hypothesis, we have $Q(sy', y')$, and so $Q(0, sy')$ by h_1 .
2. Case $x = sx'$: Since $\rho(y, x') = \max(2y - 1, 2x') < \max(2x' + 1, 2y) = \rho(sx', y)$, by the induction hypothesis, we have $Q(y, x')$, and so $Q(sx', y)$ by h_2 . $\square$

The key point in this proof conversion is the construction of ρ . In fact, ρ can be given as a *ranking function of size-change graphs* associated to the proof.

To see that, we describe how x and y change in each “Go to” in our cyclic proof, or equivalently, how the arguments of function **f** change, as the graphs G_1, G_2 in Figure 2. The graphs G_1 and G_2 correspond to the first and second “Go to” (or recursive call of **f**), respectively. For $X = x, y$ and $Y = x, y$, the arc $X \xrightarrow{\downarrow} Y$ (resp. $X \xrightarrow{\mathbb{F}} Y$) for in each graph means that Y after the corresponding “Go to” is less than (resp. less than or equal to) X before the “Go to”.

We can compose copies of G_1 and G_2 : for example, the composition $G_1 G_2 G_2$ is given in Figure 3 and this corresponds to the path in the proof tree, Case 1b $\rightarrow$ Case 2 $\rightarrow$ Case 2. It

is not difficult to see that, in any infinite composition $G_{i_1}G_{i_2}G_{i_3}\dots$ ($i_1, i_2, i_3, \dots \in \{1, 2\}$), any infinite path of arcs $\xrightarrow{\downarrow}$ and $\xrightarrow{\bar{\downarrow}}$ has infinite number of arc $\xrightarrow{\downarrow}$. This is another way to see the global trace condition.

Bipartite graphs with arcs $\xrightarrow{\downarrow}, \xrightarrow{\bar{\downarrow}}$ like G_1, G_2 are called *size-change graphs* and have been used to prove termination of programs. For a set $\mathcal{G}$ of size-change graphs, we say that $\mathcal{G}$ has *size-change termination (SCT)* condition if, in any infinite composition of graphs in $\mathcal{G}$, any infinite path of arcs has infinite number of $\xrightarrow{\downarrow}$.

The function ρ constructed in our proof by induction satisfies the following property:

Property 1. *if (i) $y_{\text{old}} \geq x_{\text{new}}$ and $y_{\text{old}} > y_{\text{new}}$ or (ii) $x_{\text{old}} > y_{\text{new}}$ and $y_{\text{old}} \geq x_{\text{new}}$ hold, then $\rho(x_{\text{old}}, y_{\text{old}}) > \rho(x_{\text{new}}, y_{\text{new}})$.*

In fact, our proof by induction is valid for any choice of ρ with this property. A function ρ with such property is called a *ranking function*. Note that the condition (i) corresponds to the graph G_1 and (ii) corresponds to G_2 , so the notion of ranking functions is defined with respect to a set of size-change graphs.

In general, we can transform any cyclic proofs with inductive definition and arithmetic into proofs with induction. This follows from the fact that there always exists a ranking function for any set of size-change graphs with SCT condition (Theorem 11).

2.2. Stage numbers

This subsection does not include technically new ideas, but aims at understanding better and intuitively how Berardi-Tatsuta's proof transformation works for general inductive definitions.

We used induction on natural numbers in the previous example. If we want to prove a proposition with inductive predicates other than natural numbers, we need to associate a natural number to each instance of the inductive predicates.

For example, suppose that we have a constant symbol l , a binary function symbol b , unary predicate symbols T, P , and inductive definition

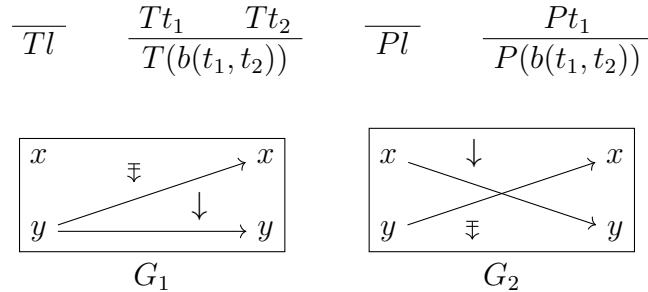


Figure 2: Size-change graphs

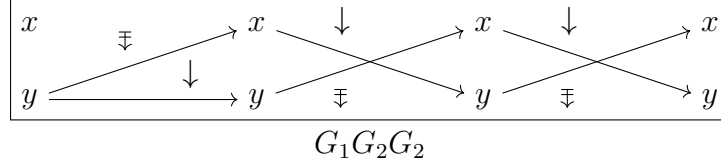


Figure 3: Composition of size-change graphs

Tt means that t is a binary tree built from branch b and leaf l like $b(b(l, l), l)$. On the other hand, Pt can hold for $t = b(b(l, s(0)), 0)$, for example.

We cyclically prove $Tt \rightarrow Pt$.

Proposition. $Tt \rightarrow Pt$.

Proof. Suppose Tt and show Pt . Since Tt holds, we have two cases:

1. Case $t = l$: By the first rule of P , we have Pl .
2. Case $t = b(t_1, t_2)$ with Tt_1, Tt_2 : By the second rule of P , it is enough to show $Tt_1 \rightarrow Pt_1$. Go to the top of the proof. $\square$

To transform this into induction, we introduce new inductive predicate symbols T' and P' with the following rules:

$$\frac{Nv}{T'(l, v)} \quad \frac{T'(t_1, v_1) \quad T'(t_2, v_2) \quad Nv_1 \quad Nv_2 \quad Nv \quad v_1 < v \quad v_2 < v}{T'(b(t_1, t_2), v)}$$

$$\frac{Nv}{P'(l, v)} \quad \frac{P'(t_1, v_1) \quad Nv_1 \quad Nv \quad v_1 < v}{P'(b(t_1, t_2), v)}$$

where we keep assuming that we have the natural number inductive predicate N with Peano axioms. The number v in $T'(t, v)$ or $P'(t, v)$ is called a *stage number* and measures the depth of a proof of $T'(t, v)$ or $P'(t, v)$ for t built from l and b . For example, Tl holds immediately from the definition, so $T'(l, 0)$ holds. To prove $T(b(b(l, l), l), l)$, we need to use the second rule of T twice, so $T'(b(b(l, l), l), 2)$ holds but we cannot prove $T'(b(b(l, l), l), 1)$.

It is not difficult to show that $Tt \leftrightarrow \exists v. T'(t, v)$ and $Pt \leftrightarrow \exists v. P'(t, v)$ by induction. Thus, it suffices to prove $T'(t, v) \wedge Nv \rightarrow \exists v'. P'(t, v')$, which means that if Tt holds with depth v , then Pt holds with some depth.

Proof. We show $\forall t. T'(t, v) \wedge Nv \rightarrow P'(t, v)$ by well-founded induction on v . Assume the statement holds for any $v' < v$. Since $T'(t, v)$ holds, we have two cases:

1. Case $t = l$: By the first rule of P' , we have $P'(l, v)$.
2. Case $t = b(t_1, t_2)$ with $T'(t_1, v_1), T'(t_2, v_2), Nv_1, Nv_2, v_1, v_2 < v$: By induction hypothesis, we have $T'(t_1, v_1) \rightarrow P'(t_1, v_1)$. The proof is completed by the second rule of P' .

□

Here, our induction works simply on v , not like the previous example where we used a ranking function.

3. Introduction to Size-Change Termination

This section is based on [11].

Let F be a finite set together with a function $\alpha: F \rightarrow \mathbb{N}$. An element f of F is called a *function name* and $\alpha(f)$ is called the *arity* of f .

For each $f \in F$, we have $\alpha(f)$ distinct symbols $x_1^f, \dots, x_{\alpha(f)}^f$, called the parameters of f . We write $Par(f)$ for the set $\{x_1^f, \dots, x_{\alpha(f)}^f\}$, and Par for $\bigcup_{f \in F} Par(f)$.

Definition 2. Let $f, g \in F$ be function names. A size-change graph from f to g , written $G: f \rightarrow g$ is a bipartite graph from $Par(f)$ to $Par(g)$ with arcs labeled with either $\downarrow$ or $\Downarrow$.

The notion of size-change graphs was introduced to prove termination of programs. In this section, we do not directly treat termination of programs, but we briefly explain how size-change graphs are constructed from a program, to provide some intuition.

Given a program, for each function definition in it, we can construct size-change graph that describe how the parameters change in each function call. Each graph $G: f \rightarrow g$ indicates that the function f with the parameters $x_1^f, \dots, x_{\alpha(f)}^f$ calls the function g with the parameters $x_1^g, \dots, x_{\alpha(g)}^g$. Each arc $x_i^f \xrightarrow{\gamma} x_j^g$ ($\gamma = \downarrow, \Downarrow$) represents a relation the value of parameter x_i^f and the value passed for parameter x_j^g of the called function g . $\gamma = \downarrow$ indicates that the value strictly decreases, and $\gamma = \Downarrow$ indicates that the value weakly decreases.

We give two examples.

Example 3. Consider the program in Figure 4 (left). Here, `if e1 then e2 else e3` is considered as a primitive operator call with arguments `e1`, `e2`, `e3`. We set $F = \{\text{rev}, \text{ra}\}$, $\alpha(\text{rev}) = 1, \alpha(\text{ra}) = 2$, and $Par(\text{rev}) = \{l\}$, $Par(\text{ra}) = \{l1, l2\}$. Then, we have the size-change graphs G_1, G_2 in Figure 5 (left).

```

rev l = ra l []
ra l1 l2 = if l1 = [] then l2
           else ra (tail l1) (app l2 [head l1])
a m n = if m=0 then n+1 else
        if n=0 then a (m-1) 1
        else a (m-1) (a m (n-1))

```

Figure 4: Examples programs

Example 4. Consider the program in Figure 4 (right). We set $F = \{a\}$, $\alpha(a) = 2$, and $Par(a) = \{m, n\}$, and we have the size-change graphs G'_1, G'_2 in Figure 5 (right). Note that `a` has three function calls in its definition, but the calls `a (m-1) 1` and `a (m-1) (a m (n-1))` have the same size-change graph G'_1 .

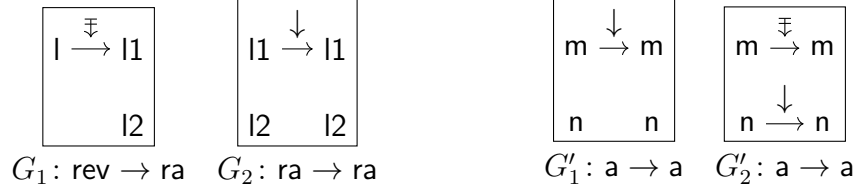


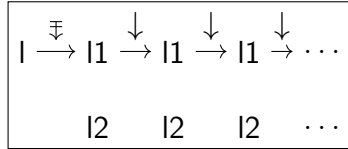
Figure 5: Examples of size-change graphs

Definition 5. Two size-change graphs G_1 and G_2 are *composable* if the target function symbol of G_1 and the source function symbol of G_2 coincide. That is, $G_1: f \rightarrow g$ and $G_2: g \rightarrow h$ for some $f, g, h \in F$. We write G_1G_2 for the directed graph that is a union of G_1 and G_2 in which, for any $p \in \text{Par}(g)$, p in G_1 and p in G_2 are identified.

Definition 6. Let $\mathcal{G}$ be a set of size-change graphs.

1. A $\mathcal{G}$ -multipath is a finite or infinite sequence $\mathcal{M} = G_1G_2\ldots$ of size-change graphs in $\mathcal{G}$ such that G_i and G_{i+1} are composable for each $i = 1, 2, \dots$. We think of $\mathcal{M}$ as a directed graph.
2. A *thread* in $\mathcal{M}$ is a (finite or infinite) directed path in $\mathcal{M}$.
3. A thread is *descending* if it contains at least one arc with $\downarrow$. It is *infinitely descending* if it contains infinitely many arcs with $\downarrow$.
4. A multipath $\mathcal{M}$ has infinite descent if some thread in $\mathcal{M}$ is infinitely descending.

For example, for size-change graphs G_1, G_2 of Example 3, we have only two $\{G_1, G_2\}$ -multipaths, $\mathcal{M}_1 = G_2G_2\ldots$ and $\mathcal{M}_2 = G_1G_2G_2\ldots$. The multipath $\mathcal{M}_2$ is drawn as:



and we can see that this has an infinitely descending thread $l \xrightarrow{\text{rev}} l1 \xrightarrow{\text{ra}} l1 \xrightarrow{\text{ra}} l1 \xrightarrow{\text{a}} \dots$

For Example 4, any sequence consisting of G_1, G_2 is a multipath.

Definition 7. Let $\mathcal{G}$ be a set of size-change graphs. We say that $\mathcal{G}$ satisfies the *SCT* (*size-change termination*) condition if every infinite $\mathcal{G}$ -multipath has infinite descent.

We can see that both Example 3 and 4 satisfy SCT.

Definition 8. Let V be a set and $>$ be a well-founded order on V . A *state* is a pair $(f, \mathbf{v})$ where f is a function name in F and $\mathbf{v}$ is an $\alpha(f)$ -tuple of elements of V . Let St denote the set of states.

Definition 9. For a size-change graph $G: f \rightarrow g$ and $\mathbf{v} = (v_1, \dots, v_{\alpha(f)})$, $\mathbf{w} = (w_1, \dots, w_{\alpha(g)})$, we write $G \models (f, \mathbf{v}) \mapsto (g, \mathbf{w})$ if (1) for each arc $x_i^f \xrightarrow{\downarrow} x_j^g$, we have $v_i > w_j$, and (2) for each arc $x_i^f \xrightarrow{\overline{\downarrow}} x_j^g$, we have $v_i \geq w_j$. Formally,

$$G \models (f, v) \mapsto (g, w) \equiv \bigwedge_{(i,j) \in I} (v_i > w_j) \wedge \bigwedge_{(i,j) \in I'} (v_i \geq w_j)$$

where I is the set of pairs (i, j) of indices such that $x_i^f \xrightarrow{\downarrow} x_j^g$ is an arc in G , and I' is the set of pairs (i, j) of indices such that $x_i^f \xrightarrow{\overline{\downarrow}} x_j^g$ is an arc in G .

Definition 10. ([12]) Let $\mathcal{G}$ be a set of size-change graphs. A $\mathcal{G}$ -ranking function is a function $\rho: St \rightarrow V$ if for any $G \in \mathcal{G}$ and $f, g \in F$, $G \models (f, \mathbf{v}) \mapsto (g, \mathbf{v}')$ implies $\rho(f, \mathbf{v}) > \rho(g, \mathbf{v}')$ where $\mathbf{v} = (v_1, \dots, v_{\alpha(f)})$ and $\mathbf{v}' = (v'_1, \dots, v'_{\alpha(g)})$.

Theorem 11. ([12]) Let $\mathcal{G}$ satisfy SCT. For any $(V, >)$, we can construct a $\mathcal{G}$ -ranking function expressed using only min, max, and lexicographic tuples of parameters and constants.

4. Introduction to Cyclic Proofs

In this section, we see the definitions of the proof system LJID, which is LJ with inductive definitions, and of the cyclic proof system CLJID^ω. Sequent calculus with inductive definitions was originally introduced for classical logic by extending LK in [1]. LJID and CLJID^ω are obtained naturally replacing LK with LJ and their definitions appear in [7].

4.1. First-order logic with inductive definitions

We introduce the language FOL_{ID}, the first-order logic with inductive definitions.

Definition 12. A first-order language with inductive predicate Σ is a set of symbols including function symbols, predicate symbols, and *inductive predicate symbols*, each of which has an associated arity.

Terms, formulas, structures, etc. are defined in the same way with the ordinary first-order logic, identifying the inductive predicate symbols and ordinary predicate symbols.

Definition 13. An *inductive definition set* Φ for a language Σ is a finite set of *production rules* of the form:

$$\frac{Q_1 \mathbf{u}_1 \dots Q_h \mathbf{u}_h \quad P_1 \mathbf{t}_1 \dots P_m \mathbf{t}_m}{P \mathbf{t}} \quad (1)$$

where $Q_1, \dots, Q_h$ are ordinary predicate symbols, $P_1, \dots, P_m, P$ are inductive predicate symbols, and $\mathbf{u}_1, \dots, \mathbf{u}_h, \mathbf{t}_1, \dots, \mathbf{t}_m, \mathbf{t}$ are tuples of terms.

Example 14. Let 0 be a constant symbol, s be a function symbol of arity 1, and N be an inductive predicate symbol of arity 1. To let Nx mean that x is a unary natural number consisting of s and 0, we can consider the following production rules:

$$\frac{}{N0} \quad \frac{Nx}{N(sx)}.$$

4.2. Proof system LJID

$$\begin{array}{c}
\frac{\phi \in \Gamma}{\Gamma \vdash \phi} \text{ (Axiom)} \quad \frac{\Gamma' \vdash \phi \quad \Gamma' \subset \Gamma}{\Gamma \vdash \phi} \text{ (Wk)} \quad \frac{\Gamma \vdash \phi \quad \Gamma', \phi \vdash \psi}{\Gamma, \Gamma' \vdash \psi} \text{ (Cut)} \quad \frac{\Gamma \vdash \phi}{\Gamma[\theta] \vdash \phi[\theta]} \text{ (Subst)} \\
\frac{\Gamma \vdash \phi \quad \Gamma, \psi \vdash \sigma}{\Gamma, \phi \rightarrow \psi \vdash \sigma} (\rightarrow L) \quad \frac{\Gamma, \phi \vdash \psi}{\Gamma \vdash \phi \rightarrow \psi} (\rightarrow R) \quad \frac{\Gamma, \phi, \psi \vdash \sigma}{\Gamma, \phi \wedge \psi \vdash \sigma} (\wedge L) \quad \frac{\Gamma \vdash \phi \quad \Gamma \vdash \psi}{\Gamma \vdash \phi \wedge \psi} (\wedge R) \\
\frac{\Gamma, \phi \vdash \sigma \quad \Gamma, \psi \vdash \sigma}{\Gamma, \phi \vee \psi \vdash \sigma} (\vee L) \quad \frac{\Gamma \vdash \phi}{\Gamma \vdash \phi \vee \psi} (\vee R1) \quad \frac{\Gamma \vdash \psi}{\Gamma \vdash \phi \vee \psi} (\vee R2) \quad \frac{}{\Gamma, \perp \vdash \phi} (\perp L) \\
\frac{\Gamma, \phi[t/x] \vdash \psi}{\Gamma, \forall x. \phi \vdash \psi} (\forall L) \quad \frac{\Gamma \vdash \phi \quad x \notin \text{FV}(\Gamma)}{\Gamma \vdash \forall x. \phi} (\forall R) \quad \frac{\Gamma, \phi \vdash \psi \quad x \notin \text{FV}(\Gamma, \psi)}{\Gamma, \exists x. \phi \vdash \psi} (\exists L) \quad \frac{\Gamma \vdash \phi[t/x]}{\Gamma \vdash \exists x. \phi} (\exists R) \\
\frac{\Gamma \vdash \phi[u/x, v/y]}{\Gamma, u = v \vdash \phi[v/x, u/y]} (=L1) \quad \frac{\Gamma, \psi[u/x, v/y] \vdash \phi}{\Gamma, \psi[v/x, u/y], u = v \vdash \phi} (=L2) \quad \frac{}{\Gamma \vdash t = t} (=R)
\end{array}$$

Figure 6: LJ=

The proof system LJID is obtained from LJ= given in Figure 6 by adding rules as follows.

- Let P be an inductive predicate and we arbitrarily order the production rules in Φ whose conclusion contains P . For the i -th production rule which is of form (1), LJID has the introduction rule PR_i

$$\frac{\Gamma \vdash Q_1 \mathbf{u}_1 \quad \dots \quad \Gamma \vdash Q_h \mathbf{u}_h \quad \Gamma \vdash P_1 \mathbf{t}_1 \quad \dots \quad \Gamma \vdash P_m \mathbf{t}_m}{\Gamma \vdash P \mathbf{t}} (PR_i)$$

- Let P be an inductive predicate symbol. We have the induction rule (Ind P) of form

$$\frac{\text{premises}}{\Gamma, P \mathbf{t} \vdash F \mathbf{t}} (\text{Ind}P)$$

for each formula $F \mathbf{t}$. In this paper, we do not explain how the premises are given. Rather, we see the case for $P = N$.

Example 15. Consider the constant symbol 0, function symbol s , predicate symbol N , and the production rules for N as introduced in Example 14. Then we have two introduction rules and an induction scheme:

$$\frac{}{\Gamma \vdash N0} (NR_1) \quad \frac{\Gamma \vdash Nx}{\Gamma \vdash N(sx)} (NR_2) \quad \frac{\Gamma \vdash F0 \quad \Gamma, Fx \vdash F(sx)}{\Gamma, Nt \vdash Ft} (\text{Ind}N)$$

Definition 16. A *derivation graph* is a 4-tuple (V, s, r, p) where

- V is a set of nodes,

- s is a total function that maps a node to a well-formed sequent,
- r is a partial function that maps a node to a rule,
- p is a partial function from $\mathbb{N} \times V$ to V (we write $p_i(v)$ for $p(i, v)$),
- and satisfies the following condition: for any $v \in V$, $p_i(v)$ are defined iff $r(v)$ is defined and it has m premises ($1 \leq i \leq m$), and

$$\frac{s(p_1(v)) \ \dots \ s(p_m(v))}{s(v)}$$

is an instance of $r(v)$.

A derivation graph can be seen as a directed graph whose vertex set is V and edge set is $\{(v, p_i(v)) \mid v \in V, p_i(v) \text{ is defined}\}$. A derivation graph is a *derivation tree* if its corresponding directed graph is a tree.

Definition 17. Let $D = (V, s, r, p)$ be a derivation tree. A *bud* of D is a node $v \in V$ such that $r(v)$ is undefined. The set of buds is written as $\text{bud}(D)$.

Definition 18. An *LJID proof* of a sequent $\Gamma \vdash \phi$ is a finite derivation tree such that $s(v) = (\Gamma \vdash \phi)$ for the root v and $\text{bud}(D) = \emptyset$.

4.3. Infinite System LJID^ω

The proof rules of LJID^ω are the rules of LJID except that for each inductive predicate P , the induction rule $\text{Ind}P$ of LJID is replaced with

$$\frac{\text{case disjunctions}}{\Gamma, P\mathbf{u} \vdash \phi} (\text{Case}P)$$

where a case disjunction is $\Gamma, \mathbf{u} = \mathbf{t}, Q_1\mathbf{u}_1, \dots, Q_h\mathbf{u}_h, P_1\mathbf{t}_1, \dots, P_m\mathbf{t}_m \vdash \phi$ for each production rule for P of form (1) subject to the restriction that for any free variable in $\mathbf{u}_1, \dots, \mathbf{u}_h, \mathbf{t}_1, \dots, \mathbf{t}_m$, or $\mathbf{t}$ does not occur in Γ, ϕ , and $\mathbf{u}$.

Example 19. For the predicate N introduced in Example 14, $\text{Case}N$ is

$$\frac{\Gamma, t = 0 \vdash \phi \quad \Gamma, t = sx, Nx \vdash \phi}{\Gamma, Nt \vdash \phi} (\text{Case}N)$$

where $x \notin FV(\Gamma \cup \phi \cup \{t\})$.

Definition 20. An LJID^ω *pre-proof* of a sequent $\Gamma \vdash \phi$ is a (possibly infinite) LJID^ω derivation tree $D = (V, s, r, p)$ whose root sequent is $\Gamma \vdash \phi$ and such that $\text{bud}(D) = \emptyset$.

Definition 21. Let (v_i) be a path in an LJID^ω pre-proof $D = (V, s, r, p)$. A *trace* following (v_i) is a sequence (τ_i) such that, for each i ,

- $\tau_i = P_i \mathbf{t}_i \in \Gamma_i$ where P_j is an inductive predicate and $s(v_i) = \Gamma_i \vdash \phi_i$,
- if $r(v_i)$ is (Subst) then $\tau_i = \tau_{i+1} \theta$ where θ is the substitution associated with the rule instance,
- if $r(v_i)$ is (=L2) with active rule $t = u$, then there exists a formula F and variables x and y such that $\tau_i = F[t/x, u/y]$ and $\tau_{i+1} = F[u/x, t/y]$,
- if $r(v_i)$ is (CaseP) for some inductive predicate P , then either $\tau_{i+1} = \tau_i$ or τ_i is the active formula of the rule instance and τ_{i+1} is a case-descendant of τ_i . In the latter case, we say that the trace progresses at this point,
- if $r(v_i)$ is not any of (Subst), (=L2), or (CaseP), then $\tau_i = \tau_{i+1}$.

Definition 22. An LJID^ω pre-proof D is an LJID^ω proof if every infinite path in D has an infinitely progressing trace following some tail of the path. (global trace condition)

4.4. Cyclic System CLJID^ω

Definition 23. Let $D = (V, s, r, p)$ be a derivation tree and let $B \in \text{bud}(D)$. A node $C \in V$ is a *companion* for B if $r(C)$ is defined and $s(C) = s(B)$ for some substitution θ .

Definition 24. A CLJID^ω pre-proof of a sequent S is a pair $(D = (V, s, r, p), R)$ where (i) D is a finite LJID^ω derivation tree with endsequent S , and (ii) $R: V \rightarrow V$ is a partial function assigning a companion to every bud node in D .

The *unfolded* tree of D is the tree each of whose bud is replaced with the subtree whose root is the companion, and repeat this replacement possibly infinitely times.

Definition 25. A CLJID^ω pre-proof is a CLJID^ω proof if its unfolded tree satisfies the global trace condition.

Definition 26. $\text{LJID} + \text{HA}$ and $\text{CLJID}^\omega + \text{HA}$ are defined to be LJID and CLJID^ω with Heyting arithmetic: we add function symbols $0, s, +, \times$ of arity $0, 1, 2, 2$, respectively, inductive predicate symbol N , and the following production rules in Example 14 and the following Heyting axioms.

$$\begin{array}{ll}
\vdash Nx \rightarrow sx \neq 0, & \vdash Nx \wedge Ny \rightarrow sx = sy \rightarrow x = y, \\
\vdash Nx \rightarrow x + 0 = x, & \vdash Nx \wedge Ny \rightarrow x + sy = s(x + y), \\
\vdash Nx \rightarrow x \times 0 = 0, & \vdash Nx \wedge Ny \rightarrow x \times sy = x \times y + x.
\end{array}$$

It is not difficult to prove (IndP) for an inductive predicate P in CLJID^ω , i.e., $\Gamma \vdash_{\text{LJID}} \phi$ implies $\Gamma \vdash_{\text{CLJID}^\omega} \phi$. Berardi-Tatsuta proved that the converse holds under arithmetic.

Theorem 27. [7] $\Gamma \vdash_{\text{CLJID}^\omega + \text{HA}} \phi$ implies $\Gamma \vdash_{\text{LJID} + \text{HA}} \phi$.

4.5. Proof Transformation

We give the outline of the proof of Theorem 27.

Suppose that we have a cyclic proof Π in $\text{CLJID}^\omega + \text{HA}$ of $\Gamma \vdash \Delta$.

1. For each inductive predicate symbol P of arity n , we introduce a new inductive predicate symbol P' of arity $n + 1$, called the *stage-number inductive predicate symbol* for P , as described in Subsection 2.2. For each production rule of form

$$\frac{Q_1 \mathbf{u}_1 \dots Q_h \mathbf{u}_h \quad P_1 \mathbf{t}_1 \dots P_m \mathbf{t}_m}{P \mathbf{t}}$$

we introduce a new production rule

$$\frac{Q_1 \mathbf{u}_1 \dots Q_h \mathbf{u}_h \quad P'_1(\mathbf{t}_1, v_1) \dots P'_m(\mathbf{t}_m, v_m) \quad v > v_1 \dots v > v_m}{P'(\mathbf{t}, v)}$$

The last argument v of P' is called the *stage number*.

2. For a sequent J , let $J^\bullet$ be the sequent obtained from J by replacing each inductive predicate $P \mathbf{t}$ with $\exists v. P'(\mathbf{t}, v)$. Also, for a sequence $\mathbf{v}$ of fresh variables, let $J_\mathbf{v}^\circ$ be the sequent obtained from J by replacing the i -th element of form $P \mathbf{t}$ with $P'(\mathbf{t}, v_i)$ for each i .

Case rules for stage-number inductive predicates produce inequalities or equalities on stage numbers in antecedents. For a path π in a $\text{CLJID} + \text{HA}$ proof, let $\text{Ineq}(\pi)$ be the collection of these (in)equalities on stage numbers produced by Case rules in π .

The following proposition roughly means that we can transform a proof of $\Gamma \vdash \phi$ into a proof of $\Gamma_\mathbf{v}^\circ \vdash \phi^\bullet$ with an extra assumption $\text{Ineq}(\pi)$ at the top of each path π from the conclusion to an assumption.

Proposition 1. (Proposition 6.3 in [7]) *For any fresh variables $\mathbf{v}$, $\Gamma \vdash_{\text{CLJID}^\omega + \text{HA}} \phi$ with assumptions $(\Gamma_i \vdash \phi_i)_{i \in I}$ without any buds implies $\Gamma_\mathbf{v}^\circ \vdash_{\text{CLJID}^\omega + \text{HA}} \phi^\bullet$ with assumptions $(\text{Ineq}(\pi_i), (\Gamma_i)_\mathbf{v}_i^\circ \vdash \phi_i)_{i \in I}$ without any buds for some fresh variables $\mathbf{v}_i$ ($i \in I$), where π_i is the path from the conclusion to the assumption $(\Gamma_i)_\mathbf{v}_i^\circ \vdash \phi_i$.*

For a $\text{CLJID}^\omega + \text{HA}$ proof Π of $\Gamma \vdash \phi$ with assumptions, we write Π° for the $\text{CLJID}^\omega + \text{HA}$ proof of $\Gamma_\mathbf{v}^\circ \vdash \phi^\bullet$ (with suitable assumptions) obtained from the above proposition.

3. For a $\text{CLJID}^\omega + \text{HA}$ proof Π without assumptions but with buds $(p_i)_{i \in \{1, \dots, l\}}$, we can cut Π into pieces of $\text{CLJID}^\omega + \text{HA}$ proofs $(\Pi_j)_{j \in \{1, \dots, k\}}$ such that the root of Π_i is a companion of Π and buds of Π are considered as assumptions of Π_i . We apply Proposition 1 to each Π_i to obtain Π_i° .
4. Now, it suffices to prove assumptions of Π_i° s by induction. This is done by proving the formula $G(x_0, \mathbf{x})$ defined as follows. Let $C_1, \dots, C_k$ be the companions in Π , and let $J_j = (\Gamma_j \vdash \phi_j)$ be the sequent $s(C_j)$ for C_j for each $j = 1, \dots, k$. We define the predicate $G_j \mathbf{x}$ as

$$G_j \mathbf{x} \equiv \forall \mathbf{z}. \mathbf{x} = \mathbf{z} \rightarrow ((\Gamma_j)_\mathbf{v}^\circ \rightarrow \phi_j^\bullet)$$

where $\mathbf{z}$ is the sequence of free variables in $(\Gamma_j)^\circ_{\mathbf{v}}$ and $\phi_j^\bullet$, and define

$$G(x_0, \mathbf{x}) \equiv \bigwedge_{j=1}^k (x_0 = j \rightarrow G_j \mathbf{x}).$$

To prove $G(x_0, \mathbf{x})$, Berardi and Tatsuta constructed a well-founded relation $>_\Pi$ such that, for any path π from a companion C_j to a bud whose companion is $C_{j'}$,

$$\text{Ineq}(\pi) \vdash_{\text{HA}} (j, \mathbf{x}) >_\Pi (j', \mathbf{y}) \quad (2)$$

holds where $\mathbf{x}, \mathbf{y}$ are the sequence of stage numbers in $J_j, J_{j'}$, respectively. Note that Property 1 is a special case of of this.

Then, it is technical but not too difficult to see that $G(x_0, \mathbf{x})$ is provable in LJID + HA by well-founded induction for such $>_\Pi$. (For more details, see Figures 3 and 4 in [6] or Figures 2 and 3 in [7])

For $>_\Pi$, Berardi and Tatsuta construct first a well-founded relation $>_\pi$ for each path π from a companion to a bud, and then define $>_\Pi$ to be the union of these relations.

In the next section, we construct another well-founded relation $>_\Pi$ satisfying (2) using Theorem 11.

5. Main Results

We construct the set of size-change graphs $\mathcal{G}$ from a cyclic proof Π in CLJID $^\omega$ + HA.

Let $q_1, \dots, q_J$ be the companions in Π and $\Gamma_j \vdash \phi_j$ be the sequent for q_j ($j = 1, \dots, J$). We think of each $j = 1, \dots, J$ as a function name for size-change graphs whose arity is defined as the length of Γ_j .

Definition 28. Let $p_1, \dots, p_I$ are the buds in Π For $i = 1, \dots, I$, suppose that $q_{c(i)}$ is the companion for p_i and q_j is an ancestor of p_i and also a companion for some bud. Suppose Γ_j (resp. $\Gamma_{c(i)}$) has inductive predicates $P_1 \mathbf{t}_1, \dots, P_n \mathbf{t}_n$ (resp. $Q_1 \mathbf{s}_1, \dots, Q_m \mathbf{s}_m$). Then, we define a size-change graph $G_i: j \rightarrow c(i)$ as follows:

- G_i has an arc $x_k^j \xrightarrow{\downarrow} x_l^{c(i)}$ iff there is a progressing trace from $P_k \mathbf{t}_k$ in Γ_j to $Q_l \mathbf{s}_l$ in $\Gamma_{c(i)}$.
- G_i has an arc $x_k^j \xrightarrow{\nrightarrow} x_l^{c(i)}$ iff there is a non-progressing trace from $P_k \mathbf{t}_k$ in Γ_j to $Q_l \mathbf{s}_l$ in $\Gamma_{c(i)}$.

We define $\mathcal{G}_\Pi$ as the set $\{G_1, \dots, G_I\}$.

Then, the following is obvious from the definition of global trace condition.

Proposition 2. *For any CLJID^ω proof, $\mathcal{G}_\Pi$ satisfies the SCT condition. Conversely, for any CLJID^ω preproof, if $\mathcal{G}_\Pi$ satisfies the SCT condition, then the preproof is a proof, i.e., satisfies the global trace condition.*

Berardi-Tatsuta defined a relation $\tilde{>}_\pi$ that describes $\text{Ineq}(\pi)$.

Definition 29. Let π be a path from $\Gamma \vdash \phi$ to $\Gamma' \vdash \phi'$ in a $\text{CLJID}^\omega + \text{HA}$ proof Π . Suppose Γ (resp. Γ') has inductive predicates $P_1 \mathbf{t}_1, \dots, P_n \mathbf{t}_n$ (resp. $Q_1 \mathbf{s}_1, \dots, Q_m \mathbf{s}_m$). We define relation $\tilde{>}_\pi$ as follows: $x \tilde{>}_\pi y$ holds if and only if $x = (x_1, \dots, x_n)$, $y = (y_1, \dots, y_m)$, $x_k > x_l$ for each pair of k, l such that there is a progressing trace from $P_k \mathbf{t}_k$ to $Q_l \mathbf{s}_l$, and $x_k = x_l$ for each pair of k, l such that there is a non-progressing trace from $P_k \mathbf{t}_k$ to $Q_l \mathbf{s}_l$.

Lemma 30. (Lemma 6.8 in [7]) *Let Π be a $\text{CLJID}^\omega + \text{HA}$ proof and π be a path from $\Gamma \vdash \phi$ to $\Gamma' \vdash \phi'$ in Π° . Suppose Γ (resp. Γ') has inductive predicates $P_1^\circ(\mathbf{t}_1, v_1), \dots, P_n(\mathbf{t}_n, v_n)$ (resp. $Q_1^\circ(\mathbf{s}_1, u_1), \dots, Q_m^\circ(\mathbf{s}_m, u_m)$). Then*

$$\text{Ineq}(\pi) \vdash_{\text{HA}} (v_1, \dots, v_n) \tilde{>}_\pi (u_1, \dots, u_m).$$

Using this lemma, we can prove our main theorem. Let ρ_Π be a $\mathcal{G}_\Pi$ -ranking function constructed by Theorem 11 for $V = \mathbb{N}$ and the ordinary order $>$ on $\mathbb{N}$.

Theorem 31. *For any path π from companion q_j to bud p_i ,*

$$\text{Ineq}(\pi) \vdash_{\text{HA}} \rho_\Pi(j, (v_1, \dots, v_n)) >_{\text{lex}} \rho_\Pi(c(i), (u_1, \dots, u_m))$$

where $>_{\text{lex}}$ is the lexicographic order on sequences of natural numbers.

Proof. By definition, the formula $G_i \models (j, \mathbf{x}) \mapsto (c(i), \mathbf{y})$ is the same with $(j, \mathbf{x}) \tilde{>}_\pi (c(i), \mathbf{y})$ except that every subformula $x_k = x_l$ in the latter is replaced with $x_k \geq x_l$ in the former. So, we have $(j, \mathbf{x}) \tilde{>}_\pi (c(i), \mathbf{y}) \vdash_{\text{HA}} (G_i \models (j, \mathbf{x}) \mapsto (c(i), \mathbf{y}))$. Also by the definition of ranking functions, we have $(G_i \models (j, \mathbf{x}) \mapsto (c(i), \mathbf{y})) \vdash_{\text{HA}} \rho(j, \mathbf{x}) > \rho(c(i), \mathbf{y})$. Then the result follows by the previous lemma. $\square$

Thus, we can define a well-founded relation $(j, \mathbf{x}) >_\Pi (j', \mathbf{y})$ for the proof transformation as $\rho_\Pi(j, \mathbf{x}) > \rho_\Pi(j', \mathbf{y})$.

6. Concluding remark

We have shown that we can naturally associate a set $\mathcal{G}_\Pi$ of size-change graphs from a $\text{CLJID}^\omega + \text{HA}$ pre-proof Π and the global trace condition of Π implies the SCT termination of $\mathcal{G}_\Pi$. Moreover, for any $\mathcal{G}_\Pi$ -ranking function ρ , one can transform a $\text{CLJID}^\omega + \text{HA}$ proof into an $\text{LJID} + \text{HA}$ proof with well-founded induction for the relation $\rho(-) > \rho(-)$ where $>$ is the lexicographic order on sequences of natural numbers.

While the proof of well-foundedness of Berardi-Tatsuta’s original relation $>_{\Pi}$ is quite complicated, well-foundedness of our relation immediately follows from well-foundedness of the lexicographic order on sequences of natural numbers. One may point out that the proof of Theorem 11, existence of ranking functions, has a similar complexity. That is true, but the size of the LJID proof transformed from a $CLJID^{\omega} + HA$ with Berardi-Tatsuta’s relation is much larger than that with our relation. This is because the proof of well-foundedness of Berardi-Tatsuta’s relation $>_{\Pi}$ is included in the transformed LJID proof, while existence of ranking functions is not.

Acknowledgement

The author thanks Makoto Tatsuta for helpful discussions and comments on this preprint. This work was supported by JSPS KAKENHI Grant Number 24K20758.

References

- [1] J. Brotherston, Cyclic proofs for first-order logic with inductive definitions, in: B. Beckert (Ed.), *Automated Reasoning with Analytic Tableaux and Related Methods*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2005, pp. 78–92.
- [2] J. Brotherston, A. Simpson, Sequent calculi for induction and infinite descent, *Journal of Logic and Computation* 21 (6) (2011) 1177–1216. doi:10.1093/logcom/exq052.
- [3] P. Martin-Löf, Hauptsatz for the intuitionistic theory of iterated inductive definitions, in: J. Fenstad (Ed.), *Proceedings of the Second Scandinavian Logic Symposium*, Vol. 63 of *Studies in Logic and the Foundations of Mathematics*, Elsevier, 1971, pp. 179–216. doi:https://doi.org/10.1016/S0049-237X(08)70847-4.
- [4] S. Berardi, M. Tatsuta, Classical system of martin-löf’s inductive definitions is not equivalent to cyclic proof system, in: *Proceedings of the 20th International Conference on Foundations of Software Science and Computation Structures - Volume 10203*, Springer-Verlag, Berlin, Heidelberg, 2017, p. 301–317.
- [5] A. Simpson, Cyclic arithmetic is equivalent to peano arithmetic, in: J. Esparza, A. S. Murawski (Eds.), *Foundations of Software Science and Computation Structures*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2017, pp. 283–300.
- [6] S. Berardi, M. Tatsuta, Equivalence of inductive definitions and cyclic proofs under arithmetic, in: *2017 32nd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, 2017, pp. 1–12. doi:10.1109/LICS.2017.8005114.
- [7] S. Berardi, M. Tatsuta, Intuitionistic podelski-rybalchenko theorem and equivalence between inductive definitions and cyclic proofs, in: C. Cîrstea (Ed.), *Coalgebraic Methods in Computer Science*, Springer International Publishing, Cham, 2018, pp. 13–33.

- 1
2
3
4
5 [8] R. N. S. Rowe, J. Brotherston, Automatic cyclic termination proofs for recursive pro-
6 cedures in separation logic, in: Proceedings of the 6th ACM SIGPLAN Conference on
7 Certified Programs and Proofs, CPP 2017, Association for Computing Machinery, New
8 York, NY, USA, 2017, p. 53–65. doi:10.1145/3018610.3018623.
9
10 [9] S. Itzhaky, H. Peleg, N. Polikarpova, R. N. S. Rowe, I. Sergey, Cyclic program synthesis,
11 in: Proceedings of the 42nd ACM SIGPLAN International Conference on Programming
12 Language Design and Implementation, PLDI 2021, Association for Computing Machin-
13 ery, New York, NY, USA, 2021, p. 944–959. doi:10.1145/3453483.3454087.
14
15 [10] T. Tsukada, H. Unno, Software model-checking as cyclic-proof search, Proc. ACM Pro-
16 gram. Lang. 6 (POPL) (jan 2022). doi:10.1145/3498725.
17
18 [11] C. S. Lee, N. D. Jones, A. M. Ben-Amram, The size-change principle for program
19 termination, SIGPLAN Not. 36 (3) (2001) 81–92. doi:10.1145/373243.360210.
20
21 [12] C. S. Lee, Ranking functions for size-change termination, ACM Trans. Program. Lang.
22 Syst. 31 (3) (apr 2009). doi:10.1145/1498926.1498928.
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65